

Thadomal Shahani Engineering College

Bandra (W.), Mumbai- 400 050.

❧ CERTIFICATE ❧

Certify that Mr./Miss Om Anirudha Shete
of Computer Engg. Department, Semester 8 with
Roll No. 2103163 has completed a course of the necessary
experiments in the subject Distributed Computing under my
supervision in the **Thadomal Shahani Engineering College**
Laboratory in the year 2024 - 2025

Teacher In-Charge

Head of the Department

Date 11/04/2025

Principal

Rodger

Experiment 2

Aim : Write a program to implement inter process communication.

Theory :

- Interprocess communication in distributed system is a process of exchanging data between two or more independent processes in a distributed environment called Inter-process communication.
- Interprocess communication on the internet provides both Datagram and stream communication.
- Characteristics :

1) Synchronous system calls

⇒ In this system calls both sender and receiver use blocking system calls to transmit the data.

2) Asynchronous system calls

⇒ In this system calls, both sender and receiver ~~waits until the message are~~ use non-blocking system call to transmit the data.

3) Reliability

⇒ It is defined as validity and integrity.

- Types :

1) Message Passing

⇒ Message passing involves process communicating by sending and receiving messages. Messages can be structured data packets containing information or commands.

2) RPC

⇒ RPC allows one process to invoke a procedure in another process, typically located on a different machine over a network.

- Advantages :

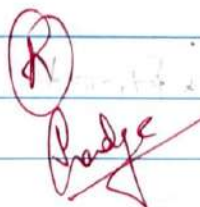
1) Enables processes to communicate with each other and share resources.

2) Can be used to implement various synchronization.

- Disadvantages :

1) Increases system complexity.

2) Can introduce security vulnerabilities.


Rudra

Code:

1. Server.java

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        int PORT = 4568;

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server is running... Waiting for client...");

            try (Socket socket = serverSocket.accept();
                BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
                PrintWriter output = new
PrintWriter(socket.getOutputStream(), true)) {

                System.out.println("Client connected: " +
socket.getInetAddress());

                String clientMessage;
                while ((clientMessage = input.readLine()) != null) {
                    System.out.println("Client: " + clientMessage);

                    if (clientMessage.equalsIgnoreCase("exit")) {
                        System.out.println("Client disconnected.");
                        break;
                    }

                    output.println("Server: " + clientMessage + " good morning");
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



```
}
```

2. Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {
    public static void main(String[] args) {
        String HOST = "127.0.0.1";
        int PORT = 4568;

        try (Socket socket = new Socket(HOST, PORT);
            PrintWriter output = new
PrintWriter(socket.getOutputStream(), true);
            BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            Scanner scanner = new Scanner(System.in)) {

            System.out.println("Connected to server...");

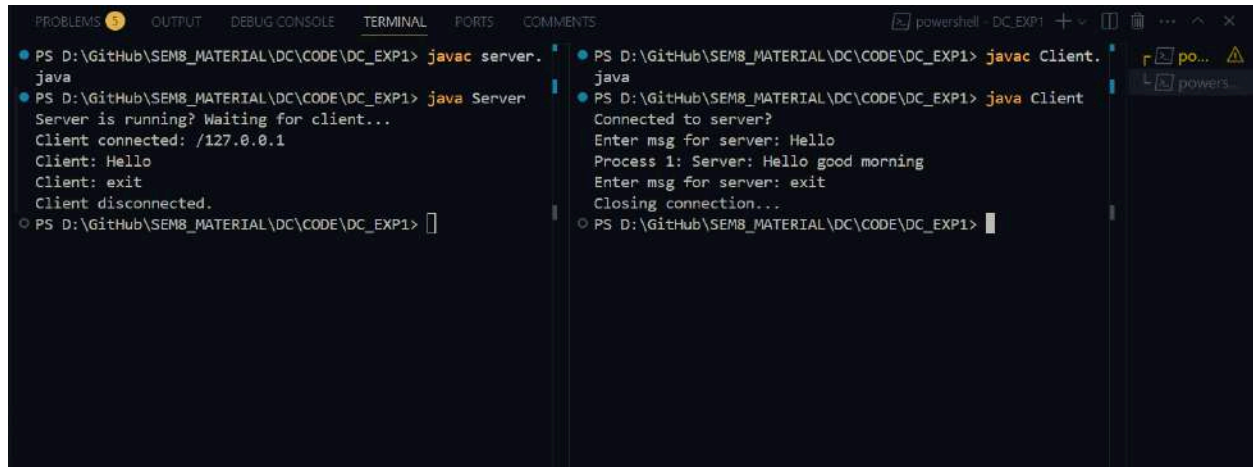
            String userMessage;
            while (true) {
                System.out.print("Enter msg for server: ");
                userMessage = scanner.nextLine();
                output.println(userMessage);

                if (userMessage.equalsIgnoreCase("exit")) {
                    System.out.println("Closing connection...");
                    break;
                }

                System.out.println("Process 1: " + input.readLine());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
}
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS powershell - DC_EXP1 + - - - - - x
● PS D:\Github\SEMB_MATERIAL\DC\CODE\DC_EXP1> javac server.
  java
● PS D:\Github\SEMB_MATERIAL\DC\CODE\DC_EXP1> java Server
Server is running? Waiting for client...
Client connected: /127.0.0.1
Client: Hello
Client: exit
Client disconnected.
○ PS D:\Github\SEMB_MATERIAL\DC\CODE\DC_EXP1>

● PS D:\Github\SEMB_MATERIAL\DC\CODE\DC_EXP1> javac Client.
  java
● PS D:\Github\SEMB_MATERIAL\DC\CODE\DC_EXP1> java Client
Connected to server?
Enter msg for server: Hello
Process 1: Server: Hello good morning
Enter msg for server: exit
Closing connection...
○ PS D:\Github\SEMB_MATERIAL\DC\CODE\DC_EXP1>
```

Experiment 2

Aim : Write a program to implement Group Communication.

Theory :

- In distributed system, it is necessary to have support for sending data to multiple receivers.
- This type of communication is called multicast communication.
- To support for ~~see~~ multicast communication, many network-level and transport-level solutions have been implemented.
- The issues in these solutions were to setup the path to disseminate the information.
- Alternative to these solutions is application level multicast techniques as peer to peer solutions are usually deployed at application layer.
- It now became easier to setup communication paths.
- Multicast messages offer good solution to build distributed system with following characteristics:

- 1) Fault tolerance based on replicated service
- Replicated service is available on group of servers. Client requests are multicast to all these servers in group. Each server performs the same operation on this request.

2) Finding discovery servers in spontaneous networking

⇒ Servers and clients can use multicast messages to find discovery services to register their interfaces of other services in the distributed system.

3) Better performance through replicated data

⇒ Replication improves performance. If in some cases, replica of data is placed in client machine then changes in data item is multicast to all the processes managing the replicas.

4) Event notification

⇒ Multicast to group to notify if some event occurs.

→ Group communication is essential in distributed systems for efficient coordination, fault tolerance, and consistency.

→ Mechanisms like multicast, publish-subscribe and gossip protocols enable's scalable and reliable communication between multiple nodes.

(R)

Prady

Code:

1. Server.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class GroupChatServer {
    private static final int PORT = 4568;
    private static final Set<PrintWriter> clientWriters =
Collections.synchronizedSet(new HashSet<>());

    public static void main(String[] args) {
        System.out.println("Group Chat Server is running...");

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("New client connected: " +
clientSocket);

                new ClientHandler(clientSocket).start();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static class ClientHandler extends Thread {
        private Socket socket;
        private PrintWriter output;
        private BufferedReader input;
        private String clientName;

        public ClientHandler(Socket socket) {
            this.socket = socket;
        }

        public void run() {
            try {
```

```

        input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        output = new PrintWriter(socket.getOutputStream(), true);

        clientWriters.add(output);

        output.println("Enter your name:");
        clientName = input.readLine();
        broadcast(clientName + " has joined the chat.");

        String message;
        while ((message = input.readLine()) != null) {
            if (message.equalsIgnoreCase("exit")) {
                break;
            }
            broadcast(clientName + ": " + message);
        }

        clientWriters.remove(output);
        broadcast(clientName + " has left the chat.");
        socket.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private void broadcast(String message) {
    synchronized (clientWriters) {
        for (PrintWriter writer : clientWriters) {
            writer.println(message);
        }
    }
    System.out.println(message);
}
}
}

```

2. Client.java


```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class GroupChatClient {
    public static void main(String[] args) {
        String HOST = "127.0.0.1";
        int PORT = 4568;

        try (Socket socket = new Socket(HOST, PORT);
            BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter output = new
PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in)) {

            System.out.print(input.readLine() + " ");
            String name = scanner.nextLine();
            output.println(name);

            new Thread(() -> {
                try {
                    String serverMessage;
                    while ((serverMessage = input.readLine()) != null) {
                        System.out.println(serverMessage);
                    }
                } catch (IOException e) {
                    System.out.println("Disconnected from chat.");
                }
            }).start();

            while (true) {
                String message = scanner.nextLine();
                if (message.equalsIgnoreCase("exit")) {
                    output.println("exit");
                    System.out.println("You left the chat.");
                    break;
                }
                output.println(message);
            }
        }
    }
}
```

```

    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

Output:

```

PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> java -v GroupChatServer.java
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> java -v GroupChatClient
Group Chat Server is running...
New client connected: Socket[addr=/127.0.0.1,port=56283,localport=4568]
Alice has joined the chat.
New client connected: Socket[addr=/127.0.0.1,port=56285,localport=4568]
Bob has joined the chat.
Alice has left the chat.
Bob has left the chat.

PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> java -v GroupChatClient
Enter your name: Alice
Alice has joined the chat.
Bob has joined the chat.
exit
You left the chat.
Disconnected from chat.
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2>

PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> java -v GroupChatClient
Enter your name: Bob
Bob has joined the chat.
Alice has left the chat.
exit
You left the chat.
Disconnected from chat.
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2>

```

Experiment 3

Aim: Write a program to simulate Clock Synchronization

Theory:

- Clock synchronization in distributed system refers to the process of ensuring that all clocks across various nodes or computers in the system are set to the same time or at least have their times closely aligned.
- In a distributed system, where multiple computers communicate and collaborate over a network, each computer typically has its own clock.
- However, due to factors such as hardware differences, network delays and clock drift, these local clocks can drift apart over time.
- Types:

1) Physical Clock Synchronization

- ⇒ In distributed systems each node operates with its clock, which can lead to time differences.
- However, the goal of physical clock synchronization is to overcome this challenge.
- This involves equipping each node with a clock that is adjusted to match UTC a recognized standard.
- By synchronizing their clocks in this way diverse systems, across the distributed landscape can maintain harmony.

2) Logical clock Synchronization

- ⇒ In distributed systems absolute time often takes a backseat to clock synchronization.
- Think of clocks as storytellers that prioritize the order of events than their exact timing.
 - These clocks enable the establishment of connections between events like weaving threads of cause and effect.
 - By bringing order and structure into play, task coordination within distributed systems become akin to a choreographed dance where steps are sequenced for execution.

3) Mutual exclusion synchronization

⇒ In the bustling symphony of distributed systems one major challenge is managing shared resources.

- Imagine multiple processes competing for access to the resources simultaneously.
- To address this issue mutual exclusion synchronization comes into play as an expert technique that reduces chaos and promote resource harmony.
- This approach relies on creating a system where different processes take turns accessing shared resources.

Prady

Code:

```
import java.util.Scanner;

class Process {
    private String name;
    private int clock;
    private Scanner scanner;
    private Process otherProcess;
    private static final Object inputLock = new Object();

    public Process(String name, int initialTime) {
        this.name = name;
        this.clock = initialTime;
        this.scanner = new Scanner(System.in);
    }

    public void setOtherProcess(Process otherProcess) {
        this.otherProcess = otherProcess;
    }

    public void sendPacket() {
        clock++;
        System.out.println(name + " sent a packet at time " + clock);
        otherProcess.receivePacket(clock);
    }

    public void receivePacket(int senderClock) {
        clock = Math.max(clock + 1, senderClock + 1);
        System.out.println(name + " received a packet at time " + clock);
    }

    public void startProcess() {
        System.out.println(name + " initial time: " + clock);

        while (true) {
            synchronized (inputLock) {
                System.out.print(name + ": Enter 'send' to send packet, 'exit' to quit: ");
                String input = scanner.nextLine().trim().toLowerCase();
            }
        }
    }
}
```


Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
powerShell - DC_EXP3 + - [ ] ... ^ x

PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP3> javac ClockSynchronization.java
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP3> java ClockSynchronization
Process 1 initial time: 0
Process 1: Enter 'send' to send packet, 'exit' to quit: Process 2 initial time: 5
send
Process 1 sent a packet at time 1
Process 2 received a packet at time 6
Process 1: Enter 'send' to send packet, 'exit' to quit: send
Process 1 sent a packet at time 2
Process 2 received a packet at time 7
Process 1: Enter 'send' to send packet, 'exit' to quit: send
Process 1 sent a packet at time 3
Process 2 received a packet at time 8
Process 1: Enter 'send' to send packet, 'exit' to quit: exit
Process 1 is exiting...
Process 2: Enter 'send' to send packet, 'exit' to quit: exit
Process 2 is exiting...
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP3> 
```

Experiment 4

Aim: Write a program to simulate Election algorithm (Bully algorithm).

Theory:

- In a distributed systems, multiple processes run independently. When the coordinator process fails, an election algorithm is used to select a new leader.

• Bully Algorithm

- The Bully algorithm selects the process with the highest ID as the leader.
- IF the current leader fails, the next highest process takes over.
- The algorithm assumes each process knows other processes IDs.
- the process with the highest ID always wins the election.
- It is called a bully algorithm because a higher-ID process can "bully" lower-ID processes by forcing them to accept its leadership.

1) Detecting Failure

- ⇒ A process detects that the leader has failed. The process initiates an election if its ID is higher than the failed leader.

2) Election process

⇒ The initiating process sends an "ELECTION" message to all higher-ID processes.

- If a higher-ID process is alive, it responds with an "OK" message and starts its own election.
- If no higher-ID process replies, the initiating process declares itself the leader.

3) Declaring the New leader

⇒ The new leader broadcasts a "COORDINATOR" message to inform all processes.

- All other process update their records to recognize the new leader.

→ For e.g.,

Given 5 processes, P_1, P_2, P_3, P_4, P_5
(Assume P_5 is current leader and fails)

Process	Action
P_3 detects failure.	starts an election.
$P_2 \rightarrow P_3, P_4$	Sends "Election" message
P_3, P_4 reply with "Ok"	They start their own election
$P_4 \rightarrow P_5$ (No response)	P_4 declares itself leader.
P_4 broadcasts "COORDINATOR"	All processes recognize P_4 as the new leader.

Code:

```
import java.util.*;

public class BullyAlgorithm {
    static int n;
    static int co;
    static int status[] = new int[100];
    static int process[] = new int[100];

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of processes:");

        n = sc.nextInt();
        int i, c, cl = 1;
        for (i = 0; i < n; i++) {
            status[i] = 1;
            process[i] = i;
        }

        boolean choice = true;
        int ch;
        do {
            System.out.println("\nChoose an option:");
            System.out.println("1. Crash Process");
            System.out.println("2. Recover Process");
            System.out.println("3. Exit");
            System.out.println(">");
            ch = sc.nextInt();
            switch (ch) {
                case 1:
                    System.out.println("Enter the process to crash:");
                    c = sc.nextInt();
                    status[c - 1] = 0;
                    cl = 1;
                    break;
                case 2:
                    System.out.println("Enter the process to recover:");
                    c = sc.nextInt();
```

```

        status[c - 1] = 1;
        break;
    case 3:
        choice = false;
        cl = 0;
        break;
    default:
        System.out.println("Invalid option. Please try
again.");
    }
    if (cl == 1) {
        System.out.println("Which process will initiate election:
");

        int ele = sc.nextInt();
        elect(ele);
    }
    System.out.println("Final Co-ordinator is " + co);
    // sc.close();
} while (choice);

}

static void elect(int ele) {
    ele = ele - 1;
    co = ele + 1;

    for (int i = 0; i < n; i++) {
        if (process[ele] < process[i]) {
            System.out.println("Election message is sent from " + (ele
+ 1) + " to " + (i + 1));
            if (status[i] == 1)
                System.out.println("Ok message is sent from " + (i +
1) + " to " + (ele + 1));
            if (status[i] == 1)
                elect(i + 1);
        }
    }
}

}

```

```
}

```

Output:

```
PS D:\Github\SEMS_MATERIAL\DC\CODE\DC_EXP4> javac BullyAlgorithm.java
PS D:\Github\SEMS_MATERIAL\DC\CODE\DC_EXP4> java BullyAlgorithm
Enter the number of processes:
5

Choose an option:
1. Crash Process
2. Recover Process
3. Exit
>
1
Enter the process to crash:
5
Which process will initiate election:
2
Election message is sent from 2 to 3
Ok message is sent from 3 to 2
Election message is sent from 3 to 4
Ok message is sent from 4 to 3
Election message is sent from 4 to 5
Election message is sent from 3 to 5
Election message is sent from 2 to 4
Ok message is sent from 4 to 2
Election message is sent from 4 to 5
Election message is sent from 2 to 5
Final Co-ordinator is 4
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
powerShell - DC_EXP4

Election message is sent from 4 to 5
Election message is sent from 2 to 5
Final Co-ordinator is 4

Choose an option:
1. Crash Process
2. Recover Process
3. Exit
>
2
Enter the process to recover:
5
Which process will initiate election:
4
Election message is sent from 4 to 5
Ok message is sent from 5 to 4
Final Co-ordinator is 5

Choose an option:
1. Crash Process
2. Recover Process
3. Exit
>
3
Final Co-ordinator is 5
PS D:\Github\SEMS_MATERIAL\DC\CODE\DC_EXP4>
```


Experiment 5

Aim : Write a program to simulate Mutual Exclusion (Raymond's Tree-Based Algorithm).

Theory :

- Mutual exclusion ensures that only one process ~~as~~ accesses a shared resources at a time in a distributed system.
- Raymond's algorithm is a distributed, tree-based approach that efficiently grants access to a shared resources while minimizing message overhead.
- Steps :

1) Requesting the token

- ⇒ A process that needs the token checks if it already holds it :-
- IF yes → It enters the critical section
- IF no → It forwards request to its parent node in the tree.

2) Token passing

- ⇒ The request propagates upward until it reaches the current token holder.
- The token holder sends the token downward through the shortest path to the requesting process.

3) Entering the critical section

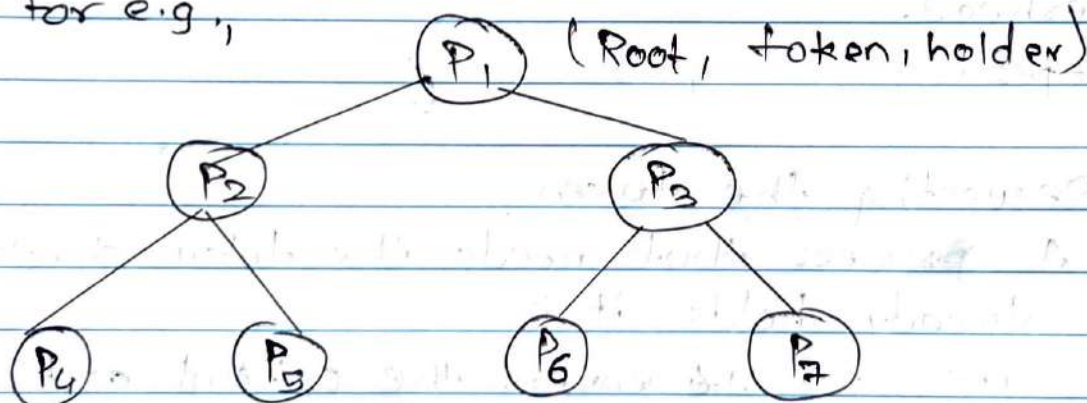
⇒ When a process receives the token, it removes itself from the queue and enters the critical section.

4) Releasing the token

⇒ After execution, the process checks its queue:

- If queue is empty → It keeps the token.
- If queue is not empty → It passes the token to the next requester in the queue.

- For e.g.,



Scenario: P₇ Requests the token

- 1) P₇ → P₃ (Inced the token)
- 2) P₃ → P₁ (P₇ needs the token)
- 3) P₁ (Token holder) → P₃ (Passes token)
- 4) P₃ → P₇ (Token reaches P₇)
- 5) P₇ enters the critical section

(A)

Prody

Code:

```

import java.util.*;

class Node {
    int nodeId;
    int parentNode;
    boolean hasToken;
    Queue<Integer> requestQueue;

    public Node(int nodeId, int parentNode, boolean hasToken) {
        this.nodeId = nodeId;
        this.parentNode = parentNode;
        this.hasToken = hasToken;
        this.requestQueue = new LinkedList<>();
    }

    public void requestToken(int requesterId, Map<Integer, Node> nodes) {
        if (hasToken) {
            System.out.println("Node " + nodeId + " sends token to Node "
+ requesterId);
            hasToken = false;
            nodes.get(requesterId).receiveToken();
        } else {
            System.out.println("Node " + nodeId + " forwards request to
Node " + parentNode);
            nodes.get(parentNode).requestToken(requesterId, nodes);
        }
    }

    public void receiveToken() {
        hasToken = true;
        System.out.println("Node " + nodeId + " received the token.");
        if (!requestQueue.isEmpty()) {
            int nextNode = requestQueue.poll();
            System.out.println("Node " + nodeId + " sends token to Node "
+ nextNode);
            hasToken = false;
        }
    }
}

```



```

}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();
        Map<Integer, Node> nodes = new HashMap<>();
        int rootNode = -1;
        for (int i = 0; i < n; i++) {
            System.out.print("Enter Node ID for node " + (i + 1) + ": ");
            int nodeId = sc.nextInt();
            System.out.print("Enter Parent Node ID for Node " + nodeId +
": ");

            int parentNode = sc.nextInt();
            System.out.print("Does this node have the token initially?
(true/false): ");
            boolean hasToken = sc.nextBoolean();
            nodes.put(nodeId, new Node(nodeId, parentNode, hasToken));
            if (hasToken)
                rootNode = nodeId;
        }

        System.out.print("Enter Node ID that requests the token: ");
        int requestingNode = sc.nextInt();
        nodes.get(requestingNode).requestToken(requestingNode, nodes);
    }
}

```

Output:


```
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXPS> javac Main.java
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXPS> java Main
Enter number of nodes: 5
Enter Node ID for node 1: 1
Enter Parent Node ID for Node 1: 3
Does this node have the token initially? (true/false): false
Enter Node ID for node 2: 2
Enter Parent Node ID for Node 2: 3
Does this node have the token initially? (true/false): false
Enter Node ID for node 3: 3
Enter Parent Node ID for Node 3: -1
Does this node have the token initially? (true/false): true
Enter Node ID for node 4: 4
Enter Parent Node ID for Node 4: 2
Does this node have the token initially? (true/false): false
Enter Node ID for node 5: 5
Enter Parent Node ID for Node 5: 2
Does this node have the token initially? (true/false): false
Enter Node ID that requests the token: 4
Node 4 forwards request to Node 2
Node 2 forwards request to Node 3
Node 3 sends token to Node 4
Node 4 received the token.
```

Experiment 6

Aim: Write a program to simulate Deadlock management in distributed systems (Banker's)

Theory:

- Deadlock occurs in a distributed system when a group of processes waits indefinitely for resources held by other processes, forming a circular wait.
- Deadlock Conditions :-
 - 1) Mutual exclusion - A resource is held by one process at a time.
 - 2) Hold and wait - A process holds a resource and waits for more.
 - 3) No preemption - Resources cannot be taken away.
 - 4) Circular wait - A cycle of waiting processes exists.

Banker's Algorithm :-

- Banker's algorithm is a deadlock avoidance technique that prevents deadlock by allocating resources safely.
- Steps :-

1) Check request validity.

→ A process requests resources. The system verifies if the request $\leq$ Need.

2) Simulate Allocation

- ⇒ The system temporarily allocates resources to the process.
- It runs a safety algorithm to check if the system remains in a safe state.

3) Grant or Deny Request

- ⇒ IF the system remains in a safe state, the request is approved.
- IF it leads to an unsafe state, the request is denied.

- Advantages :

- 1) Prevents deadlock by ensuring safe allocation.
- 2) Efficient for systems with known resource limits.
- 3) Works well for predictable workloads.

- Disadvantages :

- 1) Requires knowledge of maximum resource demand upfront.
- 2) High computational overhead in large systems.

(A)

Prady

Code:

```
import java.util.Scanner;

public class BankersAlgorithm {
    private int processes, resources;
    private int[][] max, alloc, need;
    private int[] available;

    public BankersAlgorithm(int processes, int resources) {
        this.processes = processes;
        this.resources = resources;
        max = new int[processes][resources];
        alloc = new int[processes][resources];
        need = new int[processes][resources];
        available = new int[resources];
    }

    public void input(Scanner scanner) {
        System.out.println("Enter the available resources:");
        for (int i = 0; i < resources; i++) {
            System.out.print("Resource R" + i + ": ");
            available[i] = scanner.nextInt();
        }

        System.out.println("Enter the Max resources for each process:");
        for (int i = 0; i < processes; i++) {
            System.out.print("P" + i + ": ");
            for (int j = 0; j < resources; j++) {
                max[i][j] = scanner.nextInt();
            }
        }

        System.out.println("Enter the Allocated resources for each process:");
        for (int i = 0; i < processes; i++) {
            System.out.print("P" + i + ": ");
            for (int j = 0; j < resources; j++) {
                alloc[i][j] = scanner.nextInt();
                need[i][j] = max[i][j] - alloc[i][j];
            }
        }
    }
}
```



```

        available[j] -= alloc[i][j];
    }
}

}

public boolean isSafeState() {
    boolean[] finished = new boolean[processes];
    int[] work = available.clone();
    int count = 0;

    while (count < processes) {
        boolean found = false;
        for (int i = 0; i < processes; i++) {
            if (!finished[i]) {
                boolean canExecute = true;
                for (int j = 0; j < resources; j++) {
                    if (need[i][j] > work[j]) {
                        canExecute = false;
                        break;
                    }
                }
                if (canExecute) {
                    for (int j = 0; j < resources; j++) {
                        work[j] += alloc[i][j];
                    }
                    finished[i] = true;
                    found = true;
                    count++;
                }
            }
        }
        if (!found)
            break;
    }

    for (boolean f : finished) {
        if (!f)
            return false;
    }
    return true;
}

```

```

    }

    public void displayNeedMatrix() {
        System.out.println("Need Matrix:");
        for (int i = 0; i < processes; i++) {
            System.out.print("P" + i + ": ");
            for (int j = 0; j < resources; j++) {
                System.out.print(need[i][j] + " ");
            }
            System.out.println();
        }
    }

    public void displayAvailableResources() {
        System.out.print("Resources Remaining: ");
        for (int i = 0; i < resources; i++) {
            System.out.print(available[i] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of processes: ");
        int processes = scanner.nextInt();
        System.out.print("Enter the number of resources: ");
        int resources = scanner.nextInt();

        BankersAlgorithm bankers = new BankersAlgorithm(processes,
resources);
        bankers.input(scanner);

        bankers.displayNeedMatrix();
        bankers.displayAvailableResources();

        if (bankers.isSafeState()) {
            System.out.println("System is in a SAFE state.");
        } else {
            System.out.println("System is UNSAFE. Deadlock possible!");
        }
    }
}

```

```

    }
}
}

```

Output:

```

PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP6> javac BankersAlgorithm.java
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP6> java BankersAlgorithm
Enter the number of processes: 5
Enter the number of resources: 3
Enter the available resources:
Resource R0: 10
Resource R1: 5
Resource R2: 7
Enter the Max resources for each process:
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3
Enter the Allocated resources for each process:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2
Need Matrix:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1
Resources Remaining: 3 3 2
System is in a SAFE state.
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP6>

```


Experiment 7

Aim: Write a program to simulate Load balancing algorithm.

Theory:

- Load balancing ensures even distribution of workloads across multiple servers or nodes in a distributed system.
- This prevents overloading of any single node, improving performance, fault tolerance and scalability.
- Load balancing algorithms can be classified into static and dynamic approaches:

1) Static Load balancing

- ⇒ Assigns tasks before execution based on predefined rules.
- Works well for predictable and stable workloads.
- Example: Round Robin, Randomized algorithm

2) Dynamic Load balancing

- ⇒ Adjusts resource allocation during execution based on system load.
- Requires real-time monitoring of system performance.
- For e.g., Least connection, Weighted Round Robin.

— Key characteristics :

- 1) Traffic distribution : To keep any one server from becoming overburdened, load balancers divide incoming requests evenly among several servers.
- 2) High Availability : Application's reliability and availability are improved by load balancers, which divide traffic among several servers.
- 3) Scalability : By making it simple to add servers or resources to meet growing traffic demands, load balancers enable horizontal scaling.
- 4) Optimization : Load balancers optimize resource utilization, ensuring efficient use of servers capacity and preventing bottlenecks.
- 5) Health monitoring : Load balancers often monitor the health of servers, directing traffic away from servers experiencing issues or downtime.
- 6) SSL Termination : Some load balancers can handle SSL/TLS encryption and decryption, offloading this resource-intensive task from servers.

⑧

Page

Code:

```
import java.util.*;

public class LoadBalancer {
    private Map<Integer, Integer> nodes;
    private int nextNodeId;

    public LoadBalancer() {
        nodes = new HashMap<>();
        nextNodeId = 0;
    }

    public void addNode() {
        nodes.put(nextNodeId++, 0);
        System.out.println("Added Node " + (nextNodeId - 1));
    }

    public void removeNode(int nodeId) {
        if (!nodes.containsKey(nodeId)) {
            System.out.println("Node " + nodeId + " does not exist.");
            return;
        }
        int loadToRedistribute = nodes.get(nodeId);
        nodes.remove(nodeId);
        System.out.println("Removed Node " + nodeId + ". Redistributing "
+ loadToRedistribute + " processes.");

        for (int i = 0; i < loadToRedistribute; i++) {
            addProcess();
        }
    }

    public void addProcess() {
        if (nodes.isEmpty()) {
            System.out.println("No nodes available to assign process.");
            return;
        }
        int minLoadNode = Collections.min(nodes.entrySet(),
Map.Entry.comparingByValue()).getKey();
```

```

        nodes.put(minLoadNode, nodes.get(minLoadNode) + 1);
        System.out.println("Assigned process to Node " + minLoadNode);
    }

    public void removeProcess(int nodeId) {
        if (!nodes.containsKey(nodeId) || nodes.get(nodeId) == 0) {
            System.out.println("No process to remove from Node " +
nodeId);
            return;
        }
        nodes.put(nodeId, nodes.get(nodeId) - 1);
        System.out.println("Removed process from Node " + nodeId);
    }

    public void displayNodes() {
        System.out.println("Current Load Distribution: " + nodes);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        LoadBalancer lb = new LoadBalancer();

        while (true) {
            System.out.println("\n1. Add Process");
            System.out.println("2. Remove Process");
            System.out.println("3. Add Node");
            System.out.println("4. Remove Node");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");
            int choice = scanner.nextInt();

            switch (choice) {
                case 1:
                    lb.addProcess();
                    break;
                case 2:
                    System.out.print("Enter Node ID to remove process
from: ");

                    int nodeId = scanner.nextInt();
                    lb.removeProcess(nodeId);

```



```

        break;
    case 3:
        lb.addNode();
        break;
    case 4:
        System.out.print("Enter Node ID to remove: ");
        int removeNodeId = scanner.nextInt();
        lb.removeNode(removeNodeId);
        break;
    case 5:
        System.out.println("Exiting...");
        scanner.close();
        return;
    default:
        System.out.println("Invalid choice. Try again.");
    }
    lb.displayNodes();
}
}
}

```

Output:

```

PS D:\Github\SEMB_MATERIAL\DC\CODE\DC_EXP7> javac LoadBalancer.java
PS D:\Github\SEMB_MATERIAL\DC\CODE\DC_EXP7> java LoadBalancer

1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 3
Added Node 0
Current Load Distribution: {0=0}

1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 3
Added Node 1
Current Load Distribution: {0=0, 1=0}

1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 3
Added Node 2
Current Load Distribution: {0=0, 1=0, 2=0}

```

```
PS D:\GitHub\SEMS_MATERIAL\DC\CODE\DC_EXP7> javac LoadBalancer.java
PS D:\GitHub\SEMS_MATERIAL\DC\CODE\DC_EXP7> java LoadBalancer
```

```
1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 3
Added Node 0
Current Load Distribution: {0=0}
```

```
1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 3
Added Node 1
Current Load Distribution: {0=0, 1=0}
```

```
1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 3
Added Node 2
Current Load Distribution: {0=0, 1=0, 2=0}
```

```
Current Load Distribution: {0=1, 1=1, 2=1}
```

```
1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 1
Assigned process to Node 0
Current Load Distribution: {0=2, 1=1, 2=1}
```

```
1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 4
Enter Node ID to remove: 2
Removed Node 2. Redistributing 1 processes.
Assigned process to Node 1
Current Load Distribution: {0=2, 1=2}
```

```
1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 2
Enter Node ID to remove process from: 1
```

```
Enter Node ID to remove process from: 1
Removed process from Node 1
Current Load Distribution: {0=2, 1=1}
```

```
1. Add Process
2. Remove Process
3. Add Node
4. Remove Node
5. Exit
Enter choice: 5
Exiting...
```

```
PS D:\GitHub\SEMS_MATERIAL\DC\CODE\DC_EXP7>
```

Experiment 8

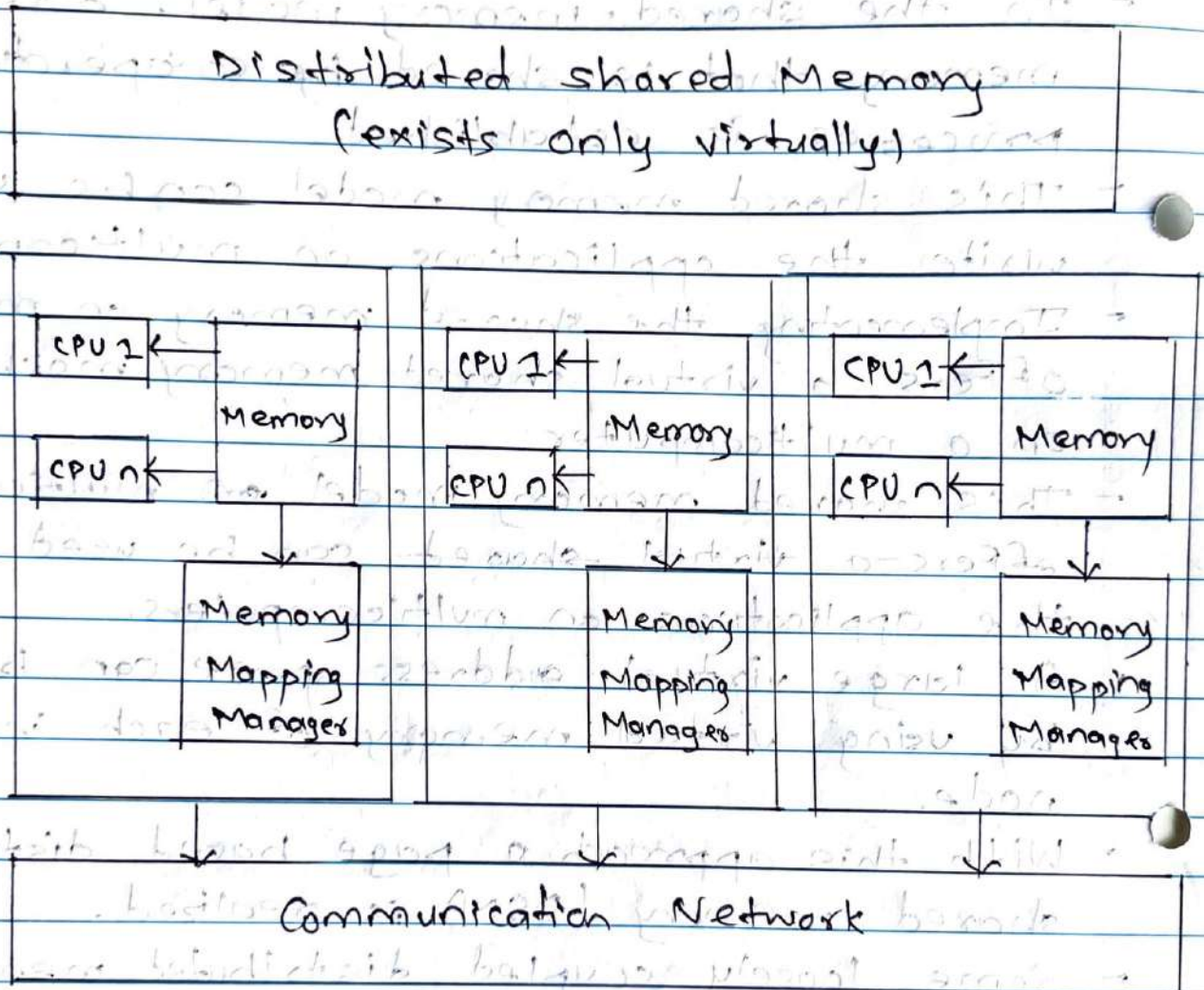
Aim : Distributed Shared Memory

Theory :

- In the shared-memory model, a region of memory that is shared by co-operating processes is established.
- This shared memory model can be used to write the applications on multicomputers.
- Implementing the shared memory on multicomputers offers a virtual shared memory machine, running on a multicomputer.
- This shared memory model on multicomputers offers a virtual shared can be used to write the applications on multicomputers.
- A large virtual address space can be created by using virtual memory of each individual node.
- With this approach a page based distributed shared memory (DSM) is realized.
- Some loosely coupled distributed memory systems have implemented a software layer on the top of message passing system in order to offer shared memory abstraction to the programmers.
- This offers the systems the illusion of physically shared memory and permit programmers to use shared memory paradigms.

• Architecture of DSM:

- Following figure shows the architecture of DSM.



- Every node in the network have one or more CPU and its local memory unit.
- These nodes are connected through high speed communication network.
- A's message passing system is present, it allows the processes running on different nodes in network to exchange messages.

- Each node contains a software memory mapping manager routine which maps local memory of its node to a virtual shared memory space.
- For this mapping purpose the shared memory space is divided into blocks.
- Data caching is supported to reduce network latency where each node's local memory is used to cache the data from virtual address space.
- The memory mapping manager of each node treats local memory as a big cache of the shared memory space for its associated CPU.
- The basic unit of caching is memory block.
- Whenever process on node access the data from memory block of shared address space, then memory mapping manager of that node handle this request.
- If block data in local memory then request will be fulfilled from local memory.
- Otherwise, network fault is generated and control is passed to operating system.
- OS then sends message to the node having that block.
- This missing block is then gets migrated from remote node to requesting process node.
- The memory mapping manager maps this block in application's address space.

- In this way data blocks keep migrating between different nodes.

• Advantages :

- 1) It is easier to implement than RPC.
- 2) DSM improves performance and efficiency by speeding up access to data.
- 3) They all share the address space so one process can easily be moved to a different machine.

• Disadvantages :

- 1) The data access is slow in DSM as compare to non-distributed.
- 2) When programming is done in DSM systems, programmers need to maintain consistency.
- 3) CPU gets slowed down, even cache memory does not aid the situation.

(A)

~~Pudge~~

Experiment 9

Aim: Distributed File system

Theory:

- File system describes how files are structured, named, accessed, used, protected and implemented.
- Files are used for permanent use of information on secondary storage.
- In distributed systems, files are available on several computers.
- Computers in distributed system can share these physically dispersed files by using distributed file system.
- Service offers particular function to client and it is a software entity running on some machines.
- Server runs service software on single machine. Client process invokes the service through some set of defined operations called as client interface.
- DFS supports the following:
 - 1) Remote Information Sharing
⇒ Any node in the system can transparently access file irrespective of its location.
 - 2) User mobility
⇒ DFS allows the user to work on different

nodes at different times without physically relocating the secondary storage devices.

3) Availability

⇒ DFS keeps multiple copies of file on different nodes. Failure of any node or copy does not effect the operation.

4) Diskless workstations

⇒ DFS provides transparent file accessing capability; hence, economical diskless workstations can be used.

- DFS provides following three types of services:

1) Storage service

⇒ DFS deals with storage and management of space on secondary storage device that is used to store files in file system.

- Files can be stored in blocks. It offers logical view of storage system by allowing operations for storing and accessing the data in them.

2) True file service

⇒ True file service supports operations on individual file. These are creating and deleting files, modifying and accessing data

from files.

3) Name service

- ⇒ Name service offers mapping between files names to file IDs. It is difficult for human to remember file IDs.
- Most file systems uses directories to carry out mapping called as directory service.
- This service supports creation and deletion of directories, carrying out adding new file to directory, deleting the existing file from directory, changing name of the file, etc.

• Features :

1) Performance User Mobility

- ⇒ DFS should permit the user to work on different nodes at different times.
- Performance should not affect if user works on node other than his node.

2) Performance

- ⇒ DFS must give performance same as centralized file system. User should not feel the need to place file explicitly to improve performance.

3) Scalability

⇒ The DFS should also supports for growth of nodes and users in network. Such growth should not lead to service loss or performance degradation of the system.

4) High Availability

⇒ DFS should continue to functional, partial failure occurs in one or more components.

→ This failure can be communication link failure, node failure and secondary storage failure.

5) High Reliability

⇒ DFS should support for reliability. It should automatically create backup copies of the critical files that can be loss in the failure.

Rodgc

Experiment 20

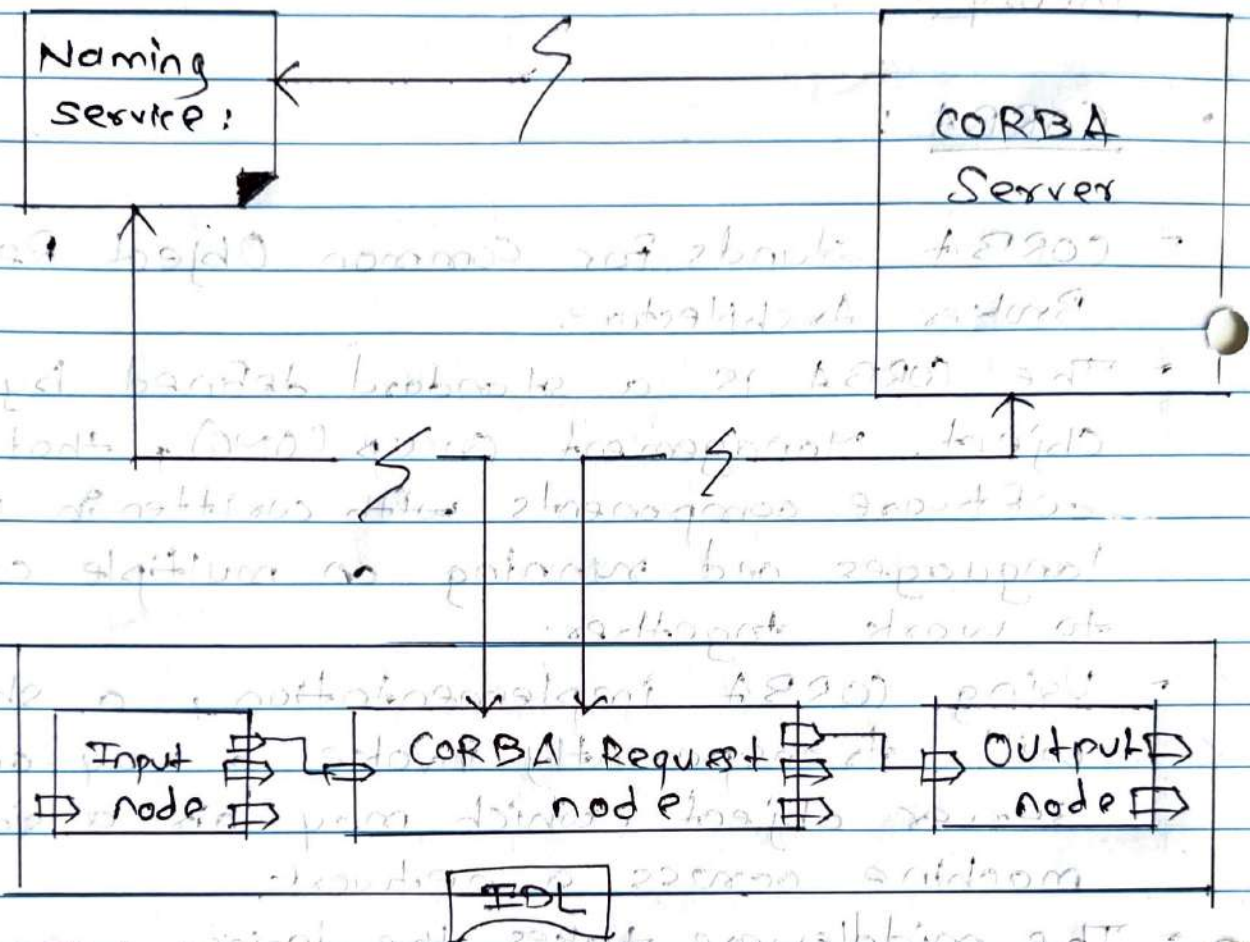
Aim : Case study : CORBA, Android stack

Theory :

• CORBA :

- CORBA stands for Common Object Request Broker Architecture.
 - The CORBA is a standard defined by the Object Management Group (OMG), that enables software components written in computer languages and running on multiple computers to work together.
 - Using CORBA implementation, a shopper will transparently invoke a way on a server object, which may be a similar machine across a network.
 - The middleware takes the decision, associated is to blame for finding an object that will implement the request, passing its parameters, invoking its methodology, and returning the results of the invocation.
 - CORBA is not associated with a particular programming language and any language with a CORBA binding can be used to call and implement objects.
- Objects* are defined in a syntax called Interface Definition Language (IDL).

→ CORBA includes four components:-



1) ORB : It handles the communication, marshaling and unmarshaling of parameters so that the parameter handling is transparent for a CORBA server and client applications.

2) CORBA Server : It creates objects and initialize them with an ORB. The server places references to the CORBA objects inside a naming service so that clients can access them.

3) Naming service : It holds references to CORBA objects.

4) CORBA Request Node : It acts as a client.

• Android stack

- The Android software stack generally consists of Linux kernel and a collection of C/C++ libraries that are exposed through an application framework, that provides services and management of the apps and run time.

- Android was created on the open-source kernel of Linux.

- Features :-

1) Security : It handles the security between the application and system.

2) Memory Management : It efficiently handles memory management thereby providing the freedom to develop our apps.

3) Network stack : It effectively handles network communication.

4) Driver model : It ensures the apps work.

- Libraries :

Running on top of kernel, the framework was developed with features.

- 1) Android runtime : It consists of core libraries of Java and ART.
- 2) Open GL : This cross-language is used to produce 2D and 3D graphics.
- 3) Webkit : This open-source web browser engine provides all the functionality to display content.
- 4) Media Framework : They allow you to play and record audio & video.
- 5) Secure Socket Layer (SSL) : They are there for Internet security.

- Applications :

It can be found at the topmost layer.

At the application layer, we write our application to be installed on this layer only.

E.g., are games, messages, contacts, etc.

Assignment - 1

Q.1 Explain issues in designing distributed system

⇒

1) Heterogeneity

⇒ The Internet enables users to access services and run applications over a heterogeneous collection of computers and networks.

- Heterogeneity applies to all of the following:-

- i) Hardware devices such as computers, tablets, mobile phones, etc.
- ii) Operating systems such as Windows, Unix, MAC, Linux, etc.
- iii) Programming languages such as Java, C/C++, Python, PHP, etc.
- iv) Different roles of software developer's, designers, system managers.

2) Transparency

⇒ Transparency is the main goal of a distributed system is to hide the actuality of physical distribution of processes and resources across several computers.

- Following types of transparency is present in distributed system:-

- i) Location: Hide where a resource is located.
- ii) Migration: Hide that a resource may move to another location.
- iii) Access: Hide differences in data representation and how a resource is accessed.

3) Scalability

⇒ As the number of users or requests increases, the system must scale accordingly without performance degradation.

- Ensuring performance across geographically dispersed locations.

4) Reliability

⇒ Using duplicate components to take over in case of failures.

- Automatically switching to a standby system when the primary system fails.

5) Security

⇒ Many of the information resources that are made available and maintained in distributed systems have a high intrinsic value to their users.

- Security for information resources has three components:-

i) Confidentiality: protection against disclosure to unauthorized individuals.

ii) Integrity: protection against alterations or corruptions.

iii) Availability: protection against interference with the means to access the resources.

Q.2 Differentiate between NOS, DOS and Middleware in the design of distributed system.

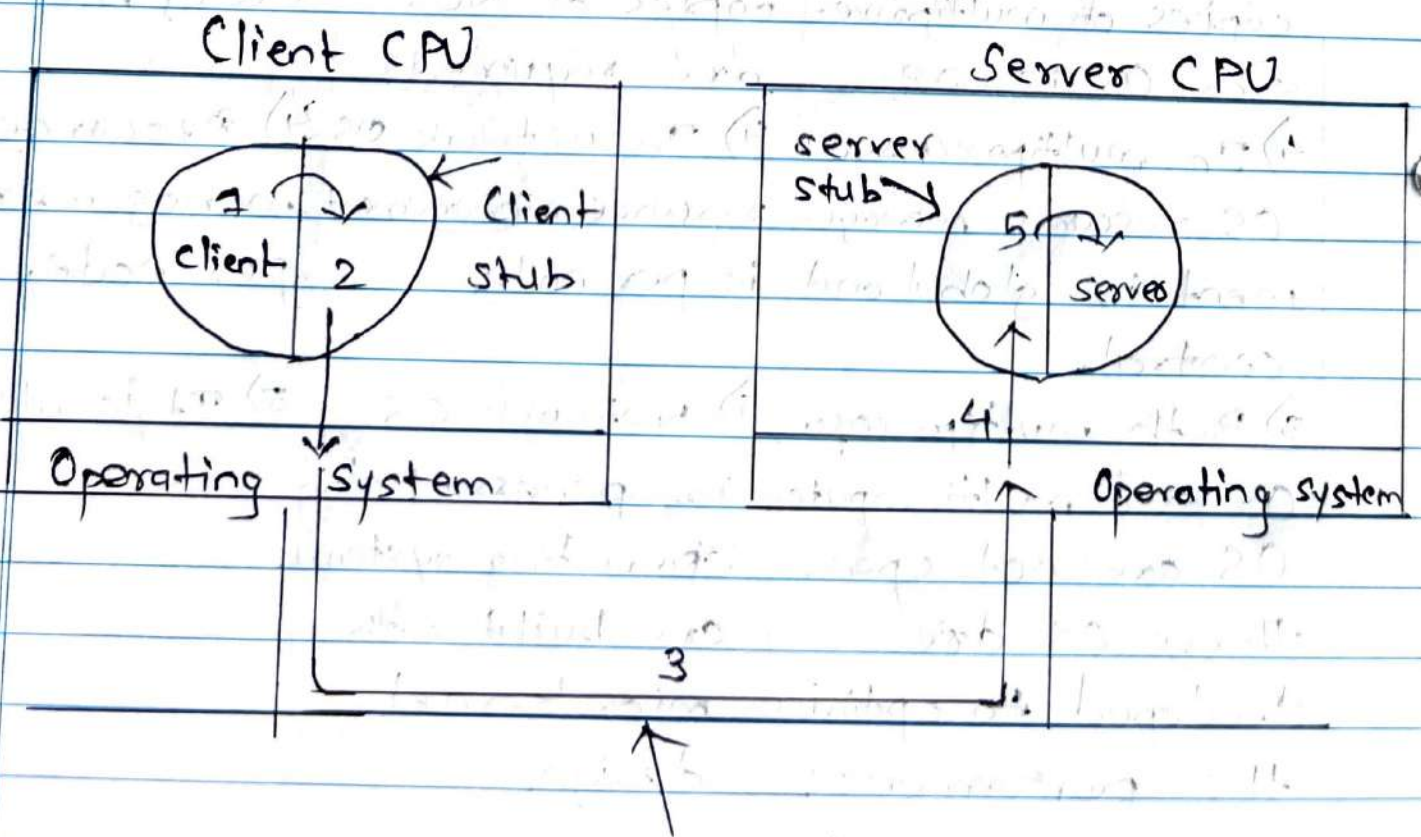
⇒

Distributed OS (DOS)	Network OS (NOS)	Middleware
1) Degree of transparency is very high for multicomputer OS. 2) Same OS is present on all nodes in both cases. 3) As multiple processors present in single machine, no of copies of multiprocessor OS is one. 4) In multiprocessor OS resource management is global and central. 5) Both multiprocessor OS and multicomputer OS are not open. These OS are developed to optimize the performance.	1) Degree of transparency is low. 2) OS on different nodes is different. 3) As multiple machines are present multiple copies of NOS are required. 4) In network OS resource management is per node. 5) Network OS is openness. Modern operating systems are built with microkernel design.	1) Degree of transparency is high. 2) OS on different nodes is different. 3) Middleware is installed on the top of NOS on every machine. 4) Resource management is per node. 5) It is also open.

Q.3 Define Remote Procedure Call. Describe the working of RPC in detail.

⇒

- Program executing on machine A calls procedure or function that is located at machine B is called as RPC.
- Although this definition seems to be simple but involves some technical issues.
- The primitives used in sending and receiving the messages do not hide communication.
- In RPC, calling program calls remote procedure by sending parameters in call and callee returns the result of procedure.
- Working:



Step 1: Client calls the client stub. As client stub is on client machine, this call is a local procedure call, and parameters pushed onto the stack in the normal manner.

Step 2: Client stub packs the parameters into a message and issue a system call to send the message. This process is called marshalling.

Step 3: Kernel sends the message from the client machine to the server machine.

Step 4: Kernel on server machine, send the incoming packet to the server stub.

Step 5: Server stub calls the server procedure.

Q.4 Write a note on Group Communication
⇒

- In distributed system, it is necessary to have support for sending data to multiple receivers.
- This type of communication is called multicast communication.
- To support multicast communication, many network level and transport-level solutions have been implemented.
- The issues in these solutions were to setup the path to disseminate the information.
- Alternative to these solutions is application level multicasting techniques as peer to peer solutions are usually deployed at application layer.
- It now became easier to setup communication paths.
- Multicast messages offers good solution to build distributed system with following characteristics.

1) Multicast Fault tolerance based on replicated service

⇒ Replicated service is available on group of servers. Client requests are multicast to all these servers in groups. Each server performs the same operation on this request. If some member of group fails, still client will be served by other active members.

2) Finding discovery servers in spontaneous networking
⇒ Servers and clients can use multicast messages to find discovery services to register their interfaces or to search the interfaces of other services in the distributed system.

3) Better performance through replicated data
⇒ Replication improves performance. If in some cases, replica of data is placed in client machine then changes in data items is multicast to all the processes managing the replicas.

4) Event notification
⇒ Multicast to group to notify if some event occurs.

(A)

Rodge

Assignment - 2

Q.1 What is the need for code migration? Explain the role of process to resource and resource to machine binding in code migration.

⇒

- Code migration refers to moving code from one machine to another in distributed system.
- This improves efficiency, resource utilization and performance.
- Need for Code Migration :-

1) Load balancing

⇒ Distribute workloads across multiple machines to prevent bottlenecks.

2) Performance Optimization

⇒ Moves execution closer to data sources to reduce network latency.

3) Fault tolerance

⇒ Allows processes to migrate to another machine in case of failures.

4) Resource Utilization

⇒ Allocates resources efficiently by moving code to machines with high processing power.

5) Scalability

⇒ Enables dynamic scaling of applications across multiple machines.

1) Process-to-Resource Binding

⇒ This refers to how a process is associated with resources. Three types of process-to-resource bindings exist.

- a) Binding by Identifier : A process is linked to a specific resource.
- b) Binding by Value : The process can access a copy of a resource.
- c) Binding by Type : The process can use any available resource that meets a requirement.

2) Resource-to-Machine Binding

⇒ This defines how a resource is linked to a physical machine.

- a) Fixed Binding : The resource is physically attached to a machine.
- b) Movable Binding : The resource can be migrated between machines.
- c) Unbound Resource : The resource is independent of machines.

Q.2 List desirable features of distributed file system. How are modifications propagated in file caching schemes?

⇒

- A distributed file system (DFS) allows multiple users to access and share files stored across multiple servers in a distributed networks.

- Key Desirable Features :-

1) Transparency

⇒ Users can access files without knowing where they are stored.

- The system automatically finds files, regardless of physical storage location.

- Multiple users can access file simultaneously.

2) Scalability

⇒ Supports large numbers of users and files.

- Handles increased load by distributing storage and computations.

3) Fault tolerance & Reliability

⇒ Redundancy & Replication ensure data availability in case of server failures.

- Automatic recovery mechanisms help maintain file integrity.

4) Performance

⇒ Local caching improves read/write speeds.
Load balancing distributes files requests efficiently.

5) Security

- ⇒ Authentication and Authorization protect against unauthorized access.
- Encryption ensures secure data transmission.

• File Modification Propagation in Caching Schemes:

- File caching is used in DFS to reduce access times by storing copies of frequently accessed files locally.

1) Write-Through Caching

- ⇒ Changes are immediately written to both the cache and the server.
- Ensures strong consistency but may cause performance overhead.
- Example: Bank transactions.

2) Write-Back Caching

- ⇒ Changes are first written to cache, then periodically flushed to the server.
- Example: Temporary files in web browser.

3) Write - Invalidate cache

- ⇒ When a process modifies a file, cached copies are invalidated on other nodes.
- Other processors must fetch the updated file before using it.
 - Example: Distributed database.

4) Write - Update caching

- ⇒ When a file is modified, updated data is broadcasted to all cached copies.
- Ensures all copies remain consistent but generates high network load.
 - Example: Collaborative editing tools.

(A)

Prody